7.3 Different exception types, different responses

Keep Food.php. We will distinguish a negative calorie count from an excessive one, then distinguish both from a wrong PHP argument type.

Introduce multiple exception classes

Replace `src/Tutorials/Class7/Dessert.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

use Exception;
use InvalidArgumentException;

class Dessert extends Food
{
    private int $calories;

    public function __construct(string $name, int $calories)
    {
        parent::__construct($name);
        $this->setCalories($calories);
    }

    public function getCalories(): int
    {
        return $this->calories;
    }

    public function setCalories(int $calories): void
    {
        if ($calories < 0) {
            throw new InvalidArgumentException('Calories cannot be negative.');
        }

        if ($calories > 5000) {
            throw new Exception('Calories exceed the maximum of 5000.');
        }

        $this->calories = $calories;
    }

    public function __toString(): string
    {
        return "Dessert: {$this->name}, {$this->calories} calories";
```

```
    }
}
```

The permitted range is 0 through 5000 inclusive. The general Exception for excessive calories temporarily follows the chapter; we will make it more specific in 7.4.

Replace `public/tutorials/class7/index.php`:

```php
<?php
declare(strict_types=1);

use App\Tutorials\Class7\Dessert;
use App\Tutorials\Class7\Food;

require_once __DIR__ . '/../../../vendor/autoload.php';

foreach ([-1, 0, 500, 5000, 5001] as $calories) {
    echo "Attempt: {$calories}", '<br>', PHP_EOL;
    try {
        $dessert = new Dessert('Berry crumble', $calories);
        echo $dessert, '<br>', PHP_EOL;
    } catch (InvalidArgumentException $e) {
        echo 'Correct the negative value: ', $e->getMessage(), '<br>', PHP_EOL;
    } catch (Exception $e) {
        echo 'Other exception: ', $e->getMessage(), '<br>', PHP_EOL;
    }
    echo 'Attempt finished', '<br>', PHP_EOL;
}

try {
    new Dessert('Berry crumble', '500');
} catch (TypeError $e) {
    echo 'Developer demonstration: wrong argument type', '<br>', PHP_EOL;
}
```

Run the CLI entry script. -1 takes the InvalidArgumentException catch; 5001 takes the general Exception catch; 0, 500, and 5000 print desserts. Every iteration prints Attempt finished. Separate test cases make it clear which value each attempt uses.

Why catch order matters

InvalidArgumentException extends Exception. PHP selects the first matching catch, so the specific catch belongs first. Temporarily swap the two catch blocks and run again: -1 now receives the general response. PHP does not try both handlers. Restore the order.

A general catch is not required after every specific catch. Leave unmatched failures to propagate when the current layer has no useful response. We use the broad catch here to compare the textbook's two cases; it will move to the request boundary later.

Exception, Error, and Throwable

```
Throwable (interface)
├── Exception
│   ├── InvalidArgumentException
│   └── RuntimeException
└── Error
    ├── TypeError
    └── ValueError
```

The string '500' in the final call breaks our int parameter contract because this calling file uses strict types. It throws TypeError before the constructor body runs. Exception would not match it. TypeError is in the separate Error hierarchy; Throwable matches both hierarchies.

The final catch is a controlled demonstration, not a suggested way to normalize form input. Fix a wrong call or validate and convert external input before calling the model. Do not hide programming defects with a fallback and claim success.

Ordinary PHP warnings do not automatically become exceptions. For example, a failed stream open may warn and return false. A catch is also unable to handle a syntax error that prevents its own file from being parsed. We will use Throwable only as a final request boundary, not promise that it can recover from every PHP failure.

Verify that the wrong-type demonstration appears and the script completes. Then identify one valid integer value, one invalid integer value, and one wrong-type value; explain which rule each violates.